

Отчёт по лабораторной работе

Сравнение алгоритмов сортировки (вариант 15)

1. Задание

Реализовать на языке C++ сортировку массива объектов типа «Запись ЗАГС» тремя алгоритмами: сортировка выбором, шейкер-сортировка и быстрая сортировка. Перегрузить операторы сравнения ($>$, $<$, $>=$, $<=$). Входные данные считываются из CSV-файла, выходные — результаты замеров времени — записываются в файл. По полученным данным построены графики зависимости времени сортировки от размера массива.

2. Структура данных

Массив данных ЗАГС — записи об актах о заключении брака. Каждая запись содержит следующие поля:

- ФИО жениха (groom_fio)
- Дата рождения жениха (groom_birth, формат DD.MM.YYYY)
- ФИО невесты (bride_fio)
- Дата рождения невесты (bride_birth, формат DD.MM.YYYY)
- Дата бракосочетания (marriage_date, формат DD.MM.YYYY)
- Номер ЗАГСа (zags_number, целое число 1–50)

Сравнение объектов выполняется последовательно по трём ключам:

- Номер ЗАГСа (приоритет 1)
- Дата бракосочетания (приоритет 2)
- ФИО жениха (приоритет 3)

3. Реализованные алгоритмы

3.1. Сортировка выбором (Selection Sort)

На каждом шаге находит минимальный элемент в неотсортированной части и помещает его на нужную позицию. Для минимизации дорогостоящих копирований строковых объектов реализована на вспомогательном массиве индексов — переставляются только целые числа, а не полные записи.

Сложность: $O(n^2)$ сравнений, $O(n)$ перестановок.

3.2. Шейкер-сортировка (Cocktail Shaker Sort)

Модификация пузырьковой сортировки: выполняет попеременные проходы слева направо (продвигает максимум вправо) и справа налево (продвигает минимум влево), сужая рабочую область. Также реализована на индексном массиве.

Сложность: $O(n^2)$ сравнений, $O(n^2)$ перестановок.

3.3. Быстрая сортировка (Quick Sort)

Рекурсивная реализация схемы Хоара. Опорный элемент выбирается из середины диапазона. Разделяет массив на две части и рекурсивно сортирует каждую. Работает непосредственно с объектами без индексного массива.

Средняя сложность: $O(n \log n)$; худший случай: $O(n^2)$.

3.4. `std::sort`

Сложность: $O(n \log n)$ гарантировано.

4. Датасет

Входные данные сгенерированы программой `generate.cpp`. Файл `zags_data.csv` содержит 100 000 записей со случайными русскими ФИО, датами и номерами ЗАГСa. Для замеров использовались срезы размером N из этого файла — всего 15 наборов от 100 до 100 000 элементов.

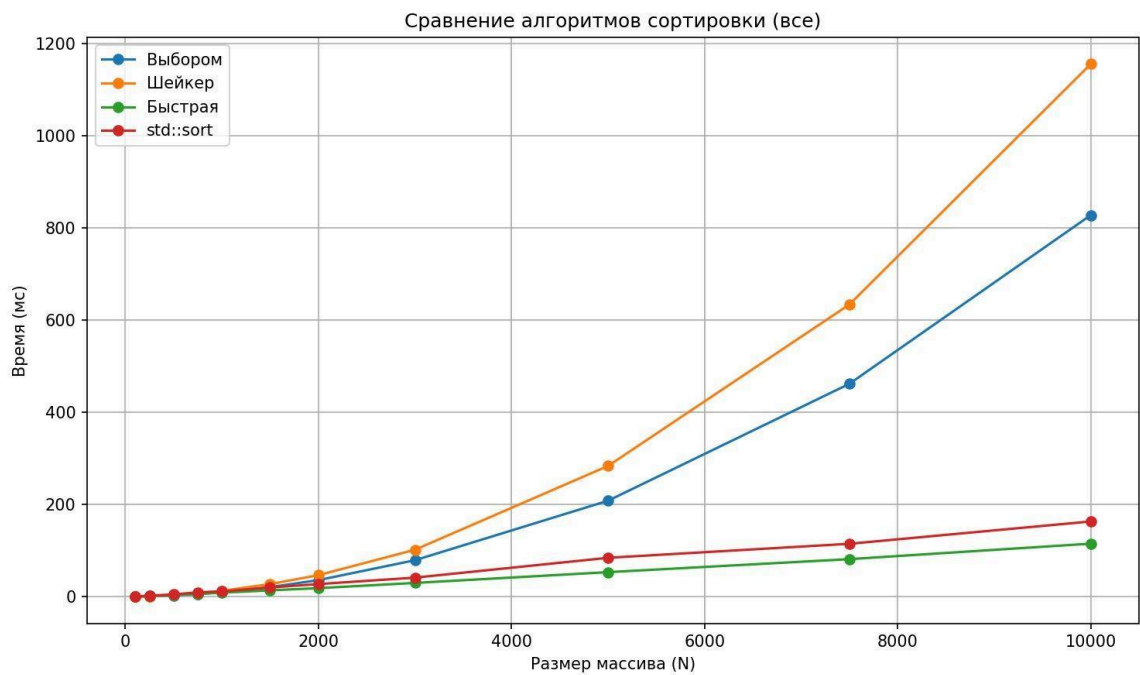
5. Результаты замеров

Время сортировки в миллисекундах для каждого алгоритма и размера массива:

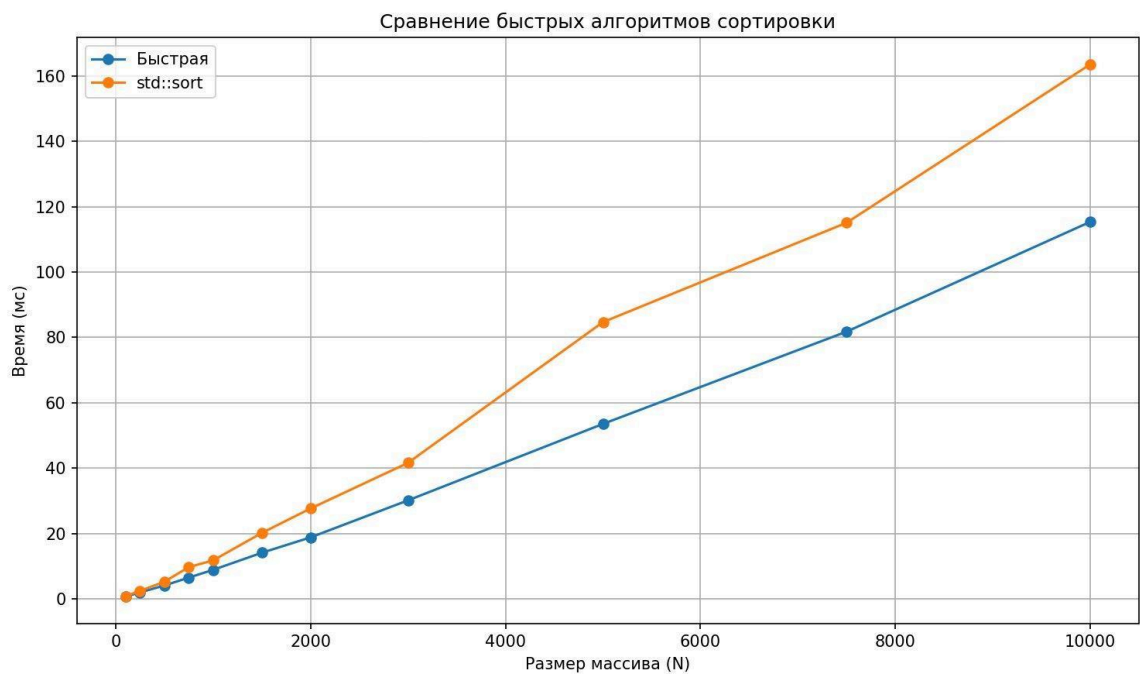
N	Выбором (мс)	Шейкер (мс)	Быстрая (мс)	<code>std::sort</code> (мс)
100	0.34	0.35	0.69	0.80
250	1.14	1.26	2.06	2.55
500	3.50	3.85	4.20	5.38
750	6.24	7.80	6.84	10.61
1 000	10.39	13.23	9.15	12.28
1 500	22.44	28.63	14.98	20.96
2 000	37.65	49.77	19.44	28.18
3 000	77.25	104.49	31.07	43.12
5 000	210.56	286.48	54.54	84.99
7 500	475.19	657.03	84.45	117.69
10 000	822.16	1124.82	118.40	163.08
15 000	1824.52	2567.64	179.09	268.62
20 000	3222.90	4599.16	241.62	362.83
50 000	20545.87	32340.36	658.63	1033.88
100 000	86983.41	164169.40	1376.19	2299.81

6. Графики

6.1. Все алгоритмы



6.2. Быстрые алгоритмы (Quick Sort и std::sort)



7. Выводы

На основании полученных данных можно сделать следующие выводы:

- Сортировка выбором и шейкер-сортировка демонстрируют квадратичный рост времени $O(n^2)$. При $N=100\,000$ они работают 87 и 164 секунды соответственно, что делает их непригодными для больших массивов.
- Шейкер-сортировка оказалась медленнее сортировки выбором примерно вдвое из-за большего числа перестановок при каждом проходе.
- Быстрая сортировка показала наилучший результат среди реализованных алгоритмов: при $N=100\,000$ время составило 1.38 секунды, что в 63 раза быстрее сортировки выбором.
- `std::sort` оказался медленнее ручной реализации быстрой сортировки примерно в 1.7 раза. Это объясняется тем, что на случайных данных схема Хоара без переключения на другие алгоритмы работает оптимально.

8. Исходный код

Репозиторий GitHub: https://github.com/AFname/MoP_zags_sort_comp